



**COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS**

ASSIGNMENT PART 3

SEMESTER 2 2025/2026

SUBJECT CODE : CSEB5223
SUBJECT : SOFTWARE CONSTRUCTION & METHODS
DUE DATE : 26 April 2026
NUMBER OF GROUP : GROUP 4
NAME OF MEMBERS :

No.	Full Name	Student ID
1.	AHMAD ILYAS BIN AHMAD IRFAN	SW01083514
2.	WAN NUR SABRINA BINTI BOKHARI	BSW01085928
3.	NUR HIDAYAH BINTI SARWANI	BSW01085937

Table of Contents

1.0 INTRODUCTION	4
1.1 PROJECT BACKGROUND	4
1.2 PROJECT OBJECTIVE	5
2.0 USE CASE AND CLASS DESIGN OF SLMS	6
2.1 USE CASE DIAGRAM.....	6
2.1.1 USE CASE DIAGRAM DESCRIPTION.....	6
2.2 CLASS DIAGRAM.....	8
.....	8
3.0 DOCUMENTATION FORMATTING	9
4.0 CODING FILE FORMATTING	10
5.0 NAMING CONVENTIONS.....	10
6.0 FUNCTION AND METHODS MODULARITY	11
7.0 CONTINUOUS INTEGRATION RULES.....	11
8.0 COURSE CLASS	12
8.1 COURSE CLASS DESIGN.....	12
8.1 ADD COURSE INPUT IMPLEMENTATION.....	14
8.1.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES	14
8.1.2 STORE MULTIPLE COURSES IN ARRAY/LIST	14
8.1.3 PREVENT ARRAY OUT-OF-BOUND ERRORS	15
8.2 SEARCH COURSE FUNCTION DEVELOPMENT	16
8.2.1 IMPLEMENT A LINEAR SEARCH FUNCTION	16
8.2.2 INPUT SEARCH COURSE CRITERIA	17
8.2.3 DISPLAY SEARCH COURSE RESULT FOR SUCCESFUL SEARCH	18
8.2.4 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH	18
8.3 EDIT COURSE FUNCTION DEVELOPMENT	19
8.3.1 IMPLEMENT SEARCH FUNCTION TO FIND THE TARGET COURSE.....	19
8.3.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT	20
COURSE CODE)	20
8.3.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH	21
8.3.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES	21
8.4 DELETE COURSE FUNCTION DEVELOPMENT	22
8.4.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE	22
8.4.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE	23
8.4.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH	23
8.5 VIEW ALL COURSE FUNCTION DEVELOPMENT	24

8.5.1 IMPLEMENT A 'VIEW ALL' FUNCTION TO DISPLAY ALL COURSES	24
8.5.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED FORMAT.....	25
9.0 STUDENT CLASS.....	26
9.1 STUDENT CLASS DESIGN	26
9.2 ADD STUDENT FUNCTION	27
9.3 SEARCH STUDENT FUNCTION	28
9.4 EDIT STUDENT FUNCTION	29
9.5 DELETE STUDENT FUNCTION	31
9.6 VIEW ALL STUDENTS FUNCTION.....	33
9.7 EXTRA INPUT HANDLING	34
10.0 COURSEMANAGER CLASS	35
10.1 COURSEMANAGER CLASS DESIGN	35
10.2 RELATIONSHIP MANAGEMENT FUNCTION DEVELOPMENT	38
10.2.1 ASSIGN COURSE TO STUDENT FUNCTION	38
10.2.2 LIST COURSES BY STUDENT FUNCTION	40
10.2.3 LIST STUDENTS BY COURSE FUNCTION	42
11.0 SYSTEM TESTING	44
11.1 SCENARIO 1	44
11.2 SCENARIO 2.....	45
11.3 SCENARIO 3.....	46
12.0 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES AND CHALLENGES	48
12.1 BEST PRACTICES APPLIED	48
12.2 CHALLENGES FACED	49
12.3 CONCLUSION.....	50
13.0 EVALUATION CRITERIA	50

1.0 INTRODUCTION

1.1 PROJECT BACKGROUND

With the rapid growth of digital technology in education, learning institutions are increasingly relying on online platforms to manage teaching and learning activities. Traditional classroom-based management methods such as manual assignment submission, physical attendance tracking, and paper-based grading are inefficient and difficult to scale.

A Student Learning Management System (SLMS) is a web-based application designed to support digital learning processes. Similar to established platforms like Moodle, an SLMS enables students, lecturers, and administrators to interact within a centralized system. It allows course management, assignment submission, grading, material distribution, and discussion forums to be handled electronically.

The proposed SLMS will serve as a simplified academic platform that manages users, courses, learning materials, assignments, submissions, and discussions. The system emphasizes proper software construction practices, clean architecture, and structured development to ensure maintainability and scalability.

This project is developed as part of the Software Construction & Maintenance course to apply theoretical knowledge into practical system design and implementation.

1.2 PROJECT OBJECTIVE

The main objective of this project is to design and develop a Student Learning Management System (SLMS) by applying proper software construction principles and structured programming practices. A Student Learning Management System is a software platform used in educational institutions to manage academic information such as courses, students, and their relationships. The system helps organize course information, maintain student records, and support academic management activities in a structured and efficient way.

In this project, the SLMS focuses on managing course profiles and student profiles, allowing users to perform operations such as creating, searching, editing, and deleting records. The system also manages the relationship between students and courses, enabling the tracking of course enrolments. The development process follows modular design, object-oriented programming principles, and proper coding standards to ensure the system is scalable, readable, and maintainable.

The specific objectives of this project are:

1. To design the overall structure of the Student Learning Management System using use case diagrams and class diagrams.
2. To develop modules that allow manual input and management of course profiles, including course name, course code, credit hour, course summary, and MS Teams link.
3. To implement a student profile management module that stores and manages student information such as student ID, name, email, and phone number.
4. To implement system functionalities such as searching, editing, deleting, and displaying records for both courses and students.
5. To manage the relationship between students and courses, allowing students to be assigned to courses and courses to maintain a list of enrolled students.
6. To apply software construction best practices, including modular programming, proper naming conventions, documentation standards, and coding standards.
7. To implement error handling and input validation to ensure the system handles invalid inputs and potential errors effectively.

8. To demonstrate the use of version control and continuous integration through Github collaboration during the development process.

Through this project, students will demonstrate their understanding of structured software planning, object-oriented design, modular programming, and disciplined software development practices in building a simplified academic management system similar to existing learning platforms such as Moodle.

2.0 USE CASE AND CLASS DESIGN OF SLMS

2.1 USE CASE DIAGRAM

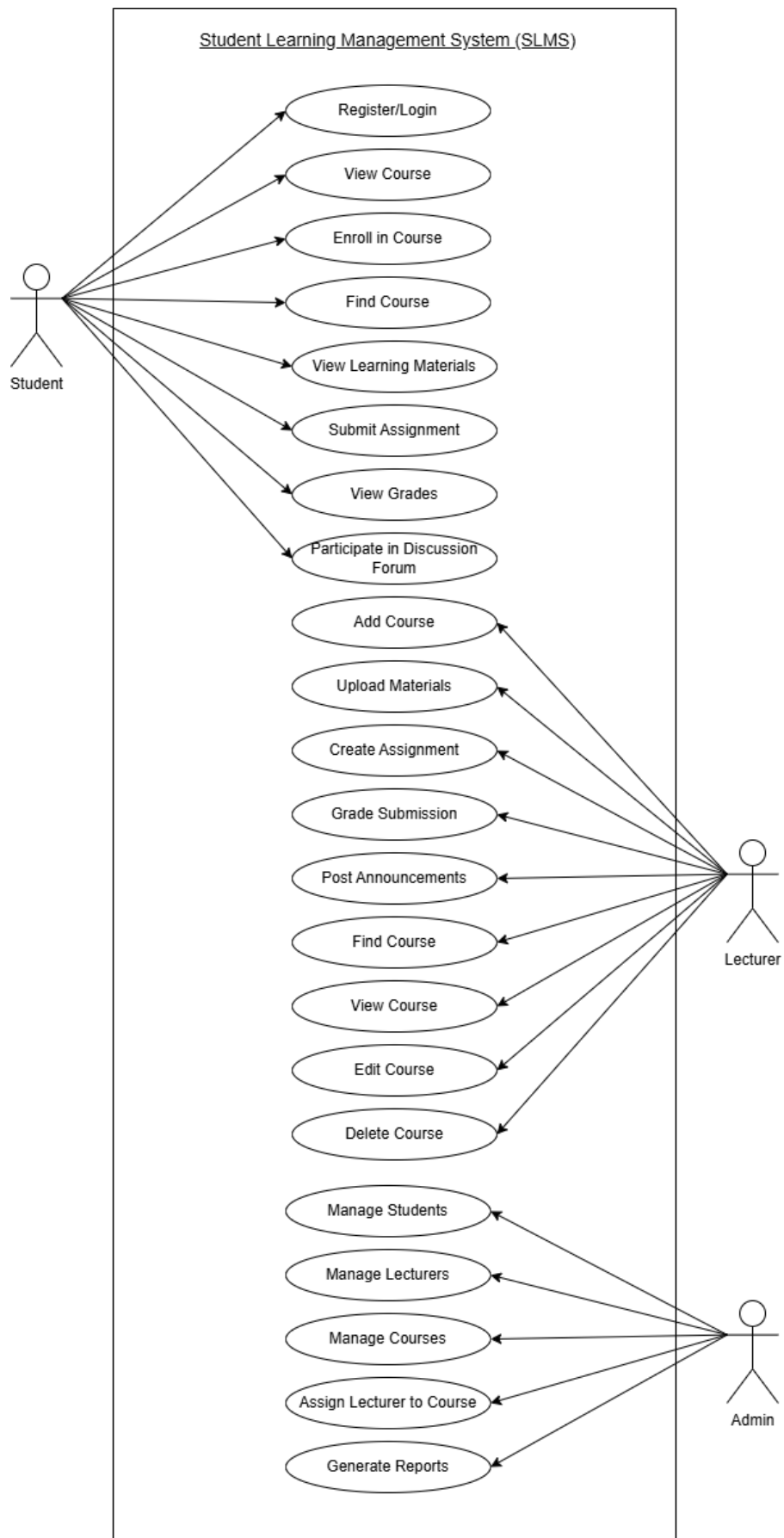
2.1.1 USE CASE DIAGRAM DESCRIPTION

A use case describes how users interact with a system to achieve a specific goal. It represents a sequence of actions between an actor and the system that leads to a useful outcome.

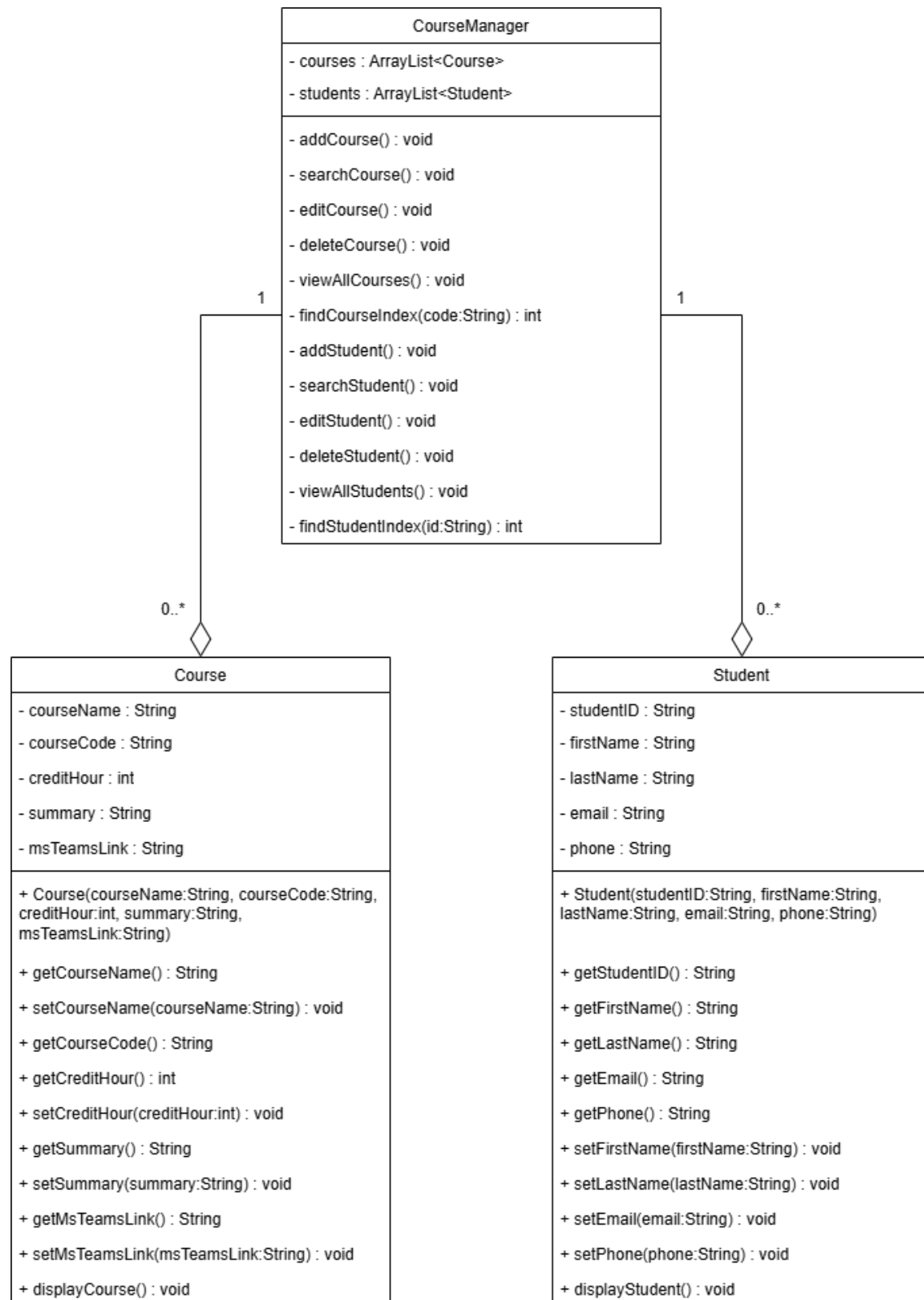
Use cases help developers understand:

- System functionality
- User requirements
- System interactions
- Scope of the system

Use case diagrams provide a high-level overview of the system's functionality and how external actors interact with it.



2.2 CLASS DIAGRAM



3.0 DOCUMENTATION FORMATTING

The SLMS documentation will follow the following format:

- Font: Times New Roman
- Font Size: 12
- Line spacing: 1.5
- Heading format:
 - 1.0 Main Heading (Bold)
 - 1.1 Subheading
- Alignment: Justified
- UML diagrams labelled properly
- Each section must include:
 - Purpose
 - Description
 - Example (if needed)

For code documentation, use:

- File header comment
- Function header comment
- Inline comments for complex logic

Example:

```
"""
```

```
File: course.py
```

```
Author: Team SLMS
```

```
Description: Handles course-related operations.
```

```
Date: 21 Feb 2026
```

```
"""
```

4.0 CODING FILE FORMATTING

- Indentation uses 4 spaces.
- Opening and closing brackets must align.
- Comments are required before functions.
- Files organised into folders.

Example:

```
if (loginSuccess) {  
    openDashboard();  
}
```

5.0 NAMING CONVENTIONS

Element	Convention	Example
Class	PascalCase	StudentAccount
Function	camelCase	submitAssignment()
Variable	snake_case	student_name
Constant	UPPER_CASE	MAX_LOGIN_ATTEMPT
File Name	lowercase	course.py

Term	Meaning
SLMS	Student Learning Management System
LMS	Learning Management System
Enrollment	Student registered in course
Submission	Assignment uploaded by student

6.0 FUNCTION AND METHODS MODULARITY

The system is designed using a modular approach where each function and method performs a single, well-defined task. Large functionalities are broken down into smaller and reusable functions to improve readability, maintainability, and scalability of the system.

For example, user authentication is separated into different functions such as input validation, credential verification, and session handling. This modular design allows easier debugging, testing, and future enhancement of the system.

Each module is designed to minimize dependency on other modules, allowing changes to be made with minimal impact on the overall system. This approach supports team collaboration, as different members can work on different modules independently.

7.0 CONTINUOUS INTEGRATION RULES

The following continuous integration rules are implemented to ensure effective collaboration and maintain code quality throughout the development of the Student Learning Management System (SLMS):

1. Each team member must develop features in a separate feature branch.
2. Direct commits to the main branch are prohibited.
3. All changes must be reviewed and approved by the team leader before merging.
4. Clear and meaningful commit messages are required for every commit.
5. Code must be tested to ensure it functions correctly before integration.
6. GitHub is used as the main platform for version control and collaboration.

These rules help reduce integration issues and improve the overall quality of the system.

Example:

Added assignment submission module.

Fixed login validation error.

8.0 COURSE CLASS

8.1 COURSE CLASS DESIGN

```
3 public class Course {  
4     // Attributes  
5     private String courseName;  
6     private String courseCode;  
7     private int creditHour;  
8     private String summary;  
9     private String msTeamsLink;
```

Explanation:

- Course is the blueprint for a course. All course-related data and functions are organized inside this

class, and every new course you add will be an object of this class. These are the data for each course. private means they are hidden from outside to protect data integrity. `courseCode` cannot be changed after creation, which ensures uniqueness

```
11 // Constructor  
12 public Course(String courseName, String courseCode, int creditHour, String summary, String msTeamsLink) {  
13     this.courseName = courseName;  
14     this.courseCode = courseCode;  
15     this.creditHour = creditHour;  
16     this.summary = summary;  
17     this.msTeamsLink = msTeamsLink;  
18 }
```

Explanation:

- Constructor is used to create a new course object with all required data.

```
20 // Getters and Setters  
21 public String getCourseName() { return courseName; }  
22 public void setCourseName(String courseName) { this.courseName = courseName; }  
23  
24 public String getCourseCode() { return courseCode; }  
25  
26 public int getCreditHour() { return creditHour; }  
27 public void setCreditHour(int creditHour) { this.creditHour = creditHour; }  
28  
29 public String getSummary() { return summary; }  
30 public void setSummary(String summary) { this.summary = summary; }  
31  
32 public String getMsTeamsLink() { return msTeamsLink; }  
33 public void setMsTeamsLink(String msTeamsLink) { this.msTeamsLink = msTeamsLink; }
```

Explanation:

- Getters are used to read or access the data, while setters are used to update or change the data (except `courseCode`). This keeps the data safe and controlled, which is a principle called encapsulation.

```
35      // Display all course attributes
36      public void displayCourse() {
37          System.out.println("Course Name      : " + courseName);
38          System.out.println("Course Code     : " + courseCode);
39          System.out.println("Credit Hour  : " + creditHour);
40          System.out.println("Summary      : " + summary);
41          System.out.println("MS Teams Link : " + msTeamsLink);
42          System.out.println("-----");
43      }
44  }
```

Explanation:

- This method is used to display all course information in a clear and organized format, such as course name, course code, credit hours, summary, and MS Teams link. It helps users easily read and understand the details of each course.
- This method is very useful for functions like View All, Search, Edit, and Delete because it allows the system to show the selected course information before performing any action. It also improves user experience by presenting the data neatly and clearly.

8.1 ADD COURSE INPUT IMPLEMENTATION

8.1.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES

```
48 // Validate credit is numeric
49 if (!creditInput.matches("\\d+")) {
50     System.out.println("Error: Credit must be a number!");
51     return;
52 }
53
54 int credit = Integer.parseInt(creditInput);
55
56 courses.add(new Course(name, code, credit, summary, link));
57 System.out.println("Course added successfully!");
58 }
59
60 public void searchCourse() {
61     System.out.print("Enter Code: ");
62     String code = sc.nextLine();
63     lastSearchedCourse = code;
64
65     int index = findCourseIndex(code);
66
67     if (index != -1) {
68         courses.get(index).displayCourse();
69     } else {
70         System.out.println("Course not found.");
71     }
72 }
```

Explanation:

- These are the fields where the user enters course details when adding a new course into the system. The user needs to fill in important information such as course name, course code, credit hours, summary, and the MS Teams link. Each field helps store complete course information, making it easier for students and lecturers to view, manage, and access the course details later.

8.1.2 STORE MULTIPLE COURSES IN ARRAY/LIST

```
8 // Store multiple courses
9 private ArrayList<Course> courses = new ArrayList<>();
10 private Scanner sc = new Scanner(System.in);
11 /**
```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 5

--- All Courses ---
Course Name      : SOFTWARE SECURITY
Course Code      : CSEB5113
Credit Hour     : 3
Summary         : SECURITY
MS Teams Link    : https://teams.microsoft.com/l/t
-----
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code      : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link    : https://teams.microsoft.com/l/t
-----

```

Explanation:

- ArrayList is used to store all courses added by the user in one collection. Each course entered into the system will be saved as an object inside the ArrayList. It can store any number of courses dynamically, which means there is no fixed limit like normal arrays. This makes the system more flexible because courses can be added or removed easily whenever needed.

8.1.3 PREVENT ARRAY OUT-OF-BOUND ERRORS

```

101 | | | | | int index = findCourseIndex(code);
102 | | | | |
103 | | | | | if (index != -1) {
104 | | | | |     System.out.println("Course found:");
105 | | | | |     courses.get(index).displayCourse();
106 | | | | | } else {
107 | | | | |     System.out.println("Course not found.");
108 | | | | | }
109 | | | | | }

```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 2
Enter Course Code to search: SCM
Course not found.

```

Explanation:

- Before accessing a course from the list, the program first checks whether the course exists at the selected position or index. This is important to make sure the user enters a valid course number. It helps prevent array out-of-bound errors, which happen when the program tries to access an index that does not exist in the list. By doing this validation, the system becomes safer, more stable, and less likely to crash.

8.2 SEARCH COURSE FUNCTION DEVELOPMENT

8.2.1 IMPLEMENT A LINEAR SEARCH FUNCTION

```

205 private int findCourseIndex(String code) {
206     for (int i = 0; i < courses.size(); i++) {
207         if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {
208             return i;
209         }
210     }
211     return -1;
212 }
213 }

```

Explanation:

- This method uses a linear search algorithm by looping through the ArrayList of courses one by one and comparing the user-entered course code with each stored course code.
- If a matching course code is found, the method returns the index of that course. If no match is found, it returns -1, which means the course does not exist in the system.

8.2.2 INPUT SEARCH COURSE CRITERIA

```
99      System.out.print("Enter Course Code to search: ");
100     String code = sc.nextLine();
101     int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: CSEB5223
Invalid input! Enter a number: 2
Enter Course Code to search: CSEB5223
```

Explanation:

- The user enters a course code as the search criterion to identify the specific course they want to find. The program then calls the linear search method, `findCourseIndex()`, which checks the list of courses one by one until a matching course code is found. If the course exists, the system can display or manage the course details using its index position. If no match is found, the program informs the user that the course is not available in the system. As a best practice, the program should validate the input first, such as making sure the course code is not empty and using a consistent format (uppercase/lowercase), to improve search accuracy and avoid errors.

8.2.3 DISPLAY SEARCH COURSE RESULT FOR SUCCESSFUL SEARCH

```
103  if (index != -1) {  
104      System.out.println("Course found:");  
105      courses.get(index).displayCourse();
```

```
Enter Course Code to search: CSEB5223  
Course found:  
Course Name      : SCM  
Course Code      : CSEB5223  
Credit Hour     : 3  
Summary         : GITHUB  
MS Teams Link    : https://teams.microsoft.com/l/1/  
-----
```

Explanation:

- If the course exists in the system, the program retrieves the corresponding course object using its index position in the ArrayList. After that, the displayCourse() method is called to present all the stored information in a clear and structured format. This includes important course attributes such as the course name, course code, credit hour, course summary, and MS Teams link.
- By displaying all these details together, it allows users to easily verify and review the full course information before making any further actions like editing or deleting.

8.2.4 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```
106  } else {  
107      System.out.println("Course not found.");  
108  }  
109  }
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 2  
Enter Course Code to search: CSEB5113  
Course not found.
```

Explanation:

- If the entered course code does not exist in the system, the program will display a message such as “Course not found.” This provides clear feedback to the user that their search was unsuccessful.
- It also ensures the system handles invalid or incorrect inputs properly without causing errors or crashing. By including this validation, the program becomes more user-friendly and reliable, as it guides the user to recheck and enter the correct course code.

8.3 EDIT COURSE FUNCTION DEVELOPMENT

8.3.1 IMPLEMENT SEARCH FUNCTION TO FIND THE TARGET COURSE

```
116 |         System.out.print("Enter Course Code to edit: ");
117 |         String code = sc.nextLine();
118 |         int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEBS223
```

Explanation:

- The system first asks the user to enter the course code of the course that needs to be edited.
- The program then calls the linear search method `findCourseIndex()` to locate the course in the `ArrayList`.
- If the course exists, its index will be returned for further editing.

8.3.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT COURSE CODE)

```
121 Course c = courses.get(index);
122 System.out.println("Editing Course: " + c.getCourseCode());
123
124 System.out.print("New Course Name: ");
125 c.setCourseName(sc.nextLine());
126
127 int credit;
128 while (true) {
129     System.out.print("New Credit Hour: ");
130     if (sc.hasNextInt()) {
131         credit = sc.nextInt();
132         sc.nextLine();
133         if (credit > 0) break;
134         else System.out.println("Credit hour must be positive!");
135     } else {
136         System.out.println("Invalid input! Enter a number.");
137         sc.next();
138     }
139 }
140 c.setCreditHour(credit);
141
142 System.out.print("New Summary: ");
143 c.setSummary(sc.nextLine());
144
145 System.out.print("New MS Teams Link: ");
146 c.setMsTeamsLink(sc.nextLine());
```

```
Editing Course: CSEB5223
New Course Name: SOFTWARE CONSTRUCTION & METHOD
New Credit Hour: 3
New Summary: GITHUB AND CODING
New MS Teams Link: https://teams.microsoft.com/1
```

Explanation:

- Once the course is found, the program retrieves the course object from the ArrayList.
- The user is allowed to edit all course attributes, including:
 - Course Name

- Credit Hour
- Course Summary
- MS Teams Link
- However, the course code cannot be edited because it acts as a unique identifier for each course.

8.3.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```

150 } else {
151     System.out.println("Course not found.");
152 }
153 }
```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEB5123
Course not found.
```

Explanation:

- If the course code entered by the user does not exist, the system will display “Course not found.”
- This ensures the program handles invalid input without causing system errors.

8.3.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES

```

148 System.out.println("Course updated successfully!");
149 c.displayCourse();
```

```

Course updated successfully!
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code      : CSEBS223
Credit Hour      : 3
Summary          : GITHUB AND CODING
MS Teams Link    : https://teams.microsoft.com/l/team/19'
-----

```

Explanation:

- After editing the course information, the system displays the updated course details. This allows the user to verify that the changes were successfully applied.
-

8.4 DELETE COURSE FUNCTION DEVELOPMENT

8.4.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE

```

160      System.out.print("Enter Course Code to delete: ");
161      String code = sc.nextLine();
162      int index = findCourseIndex(code);

```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 4
Enter Course Code to delete: CSEB3123

```

Explanation:

- The system asks the user to enter the course code of the course that needs to be deleted.
- The program uses the findCourseIndex() method (linear search) to locate the course in the ArrayList.
- If the course exists, the program retrieves its index for further action.

8.4.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE

```
165 | | | | Course c = courses.get(index);
166 | | | | System.out.println("Course found:");
167 | | | | c.displayCourse();
168 | | | |
169 | | | | System.out.print("Confirm deletion? (Y/N): ");
170 | | | | String confirm = sc.nextLine();
```

```
Course found:
Course Name      : SOFTWARE ENGINEERING
Course Code      : CSEB3123
Credit Hour     : 3
Summary          : SDLC
MS Teams Link    : https://teams.microso:
-----
Confirm deletion? (Y/N): Y
```

Explanation:

- If the course is found, the system displays the complete course attributes using the displayCourse() method.
- This allows the user to verify the correct course before deleting it.
- The program then asks for confirmation (Y/N) before proceeding with deletion.

8.4.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```
180 | | | | } else {
181 | | | |     System.out.println("Course not found.");
182 | | | | }
183 | | | | }
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 4
Enter Course Code to delete: CSEB5113
Course not found.
```

Explanation:

- If the entered course code does not exist, the system displays the message “Course not found.” This prevents the program from attempting to delete a non-existing course.

8.5 VIEW ALL COURSE FUNCTION DEVELOPMENT

8.5.1 IMPLEMENT A ‘VIEW ALL’ FUNCTION TO DISPLAY ALL COURSES

```
46 | | | | | case 5: viewAllCourses(); break;
```



```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 5
```

Explanation:

- The system provides a menu option (5 – View All Courses) that acts like a “View All” button.
- When the user selects this option, the program calls the viewAllCourses() method.
- This method displays all courses currently stored in the system.

8.5.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED FORMAT

```
188 private void viewAllCourses() {  
189     if (courses.isEmpty()) {  
190         System.out.println("No courses available.");  
191     } else {  
192         System.out.println("\n--- All Courses ---");  
193         for (Course c : courses) {  
194             c.displayCourse();  
195         }  
196     }  
197 }
```

```
--- All Courses ---  
Course Name      : SOFTWARE SECURITY  
Course Code      : CSEBS113  
Credit Hour     : 3  
Summary          : SECURITY  
MS Teams Link    : https://teams.microsoft.com/l/tea  
-----  
Course Name      : SOFTWARE CONSTRUCTION & METHOD  
Course Code      : CSEBS223  
Credit Hour     : 3  
Summary          : GITHUB AND CODING  
MS Teams Link    : https://teams.microsoft.com/l/tea  
-----
```

Explanation:

- This method displays all courses stored in the ArrayList. It first checks whether the list is empty.
- If courses exist, the system loops through the list and calls displayCourse() for each course.
- The displayCourse() method ensures the information is shown in a clear and organized format.

9.0 STUDENT CLASS

9.1 STUDENT CLASS DESIGN

- **Attributes:**

```
* This class includes:
* - Constructor to create a Student object
* - Getter methods to access attributes
* - Setter methods to modify editable attributes
* - displayStudent method to show student information in a clear format
*/
public class Student {

    // Attributes
    private String studentID; // Unique student identifier, cannot be changed after creation
    private String firstName; // Student's first name
    private String lastName; // Student's last name
    private String email; // Student's email address
    private String phone; // Student's phone number
```

- studentID (String) – Unique, immutable identifier for each student.
- firstName (String) – Student’s first name.
- lastName (String) – Student’s last name.
- email (String) – Student’s email address.
- phone (String) – Student’s contact number.

- **Methods:**

- **Getters:** Allow other classes (e.g., CourseManager) to access private attributes.
- **Setters:** Allow modification of firstName, lastName, email, phone but not studentID.
- **displayStudent():** Prints all student attributes in a clear and organized format.

9.2 ADD STUDENT FUNCTION

The system allows users to manually enter student information.

```
// Validate all inputs at once
if (id.isBlank() || fname.isBlank() || lname.isBlank() || email.isBlank() || phone.isBlank()) {
    System.out.println("Student not added! All fields must be filled.");
} else {
    // Only add if all fields are valid
    students.add(new Student(id, fname, lname, email, phone));
    System.out.println("Student added successfully!");
}
```

Explanation:

- The user enters student details including Student ID, First Name, Last Name, Email, and Phone Number to create a new student record. The system then performs input validation to check whether any field is left empty. If any required field is blank, the system will display the message “Student not added! All fields must be filled.”
- Ensure complete and accurate data entry. However, if all fields are properly filled, the system will confirm with the message “Student added successfully!” and proceed to store the student object in an ArrayList. This allows multiple student records to be saved dynamically without a fixed limit, making the system more flexible and scalable for managing student data.

- **Successfully added:**

```
--- SLMS SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Add Student
7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 6
Enter Student ID: BSW01085928
Enter First Name: Wan Nur
Enter Last Name: Sabrina
Enter Email: sabrina@gmail.com
Enter Phone: 0123456678
Student added successfully!
```

- **Students fail to fill all fields:**

```
Choose an option: 6
Enter Student ID:
Enter First Name: wan
Enter Last Name: sabrina
Enter Email: sabrina@gmail.com
Enter Phone: 0165766482
Student not added! All fields must be filled.
```

9.3 SEARCH STUDENT FUNCTION

The search function allows users to find a student using **Student ID**.

```
private void searchStudent() {
    System.out.print("Enter Student ID to search: ");
    String id = sc.nextLine();

    int index = findStudentIndex(id); // search for student

    if (index != -1) {
        System.out.println("Student found:");
        students.get(index).displayStudent(); // Show student details
    } else {
        System.out.println("Student not found."); // not found
    }
}
```

Explanation:

- User enters a student ID.
- The system uses linear search with equalsIgnoreCase() → search is case-insensitive.
- If found → displays all student information using displayStudent().
- If not found → displays: "Student not found."
- Search is case-insensitive, so entering lowercase/uppercase IDs works.
- Example: searching for bsw01085998 successfully finds the student.

- **Student was found:**

```
Choose an option: 6
Enter Student ID: BSW01085998
Enter First Name: Ilyas
Enter Last Name: Ahmad
Enter Email: ilyass@gmail.com
Enter Phone: 0198765543
Student added successfully!

7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 7
Enter Student ID to search: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Ahmad
Email       : ilyass@gmail.com
Phone       : 0198765543
-----
```

- **Student was not found:**

```
Choose an option: 7
Enter Student ID to search: BSW01098765
Student not found.
```

9.4 EDIT STUDENT FUNCTION

- User enters Student ID.
- The system performs a linear search to locate the student.

If found:

The user can edit:

- FirstName
- LastName
- Email
- phone

The studentID cannot be edited because it is a unique identifier. After editing, the system displays the updated student information to confirm the changes.

If student is not found:

```
Choose an option: 7
Enter Student ID to search: SW03
Student not found.
```

Outputs:

- **Before Edit:**

```
Choose an option: 6
Enter Student ID: BSW01085973
Enter First Name: Hidayah
Enter Last Name: Sarwani
Enter Email: dayah@gmail.com
Enter Phone: 0167542987
Student added successfully!
```

- **After Edited:**

```
Choose an option: 8
Enter Student ID to edit: BSW01085973
New First Name: Nur Dayah
New Last Name: Sarwani
New Email: dayah@gmail.com
New Phone: 0167542987
Student updated successfully.
Student ID   : BSW01085973
Name        : Nur Dayah Sarwani
Email       : dayah@gmail.com
Phone       : 0167542987
-----
```

- **Student Not Found:**

```
Choose an option: 8
Enter Student ID to edit: BSW01085974
Student not found.
```

Good Practice Observed:

- Clear separation of ID (immutable) and editable fields.
- Updated info displayed for verification.

9.5 DELETE STUDENT FUNCTION

```
private void deleteStudent() {
    System.out.print("Enter Student ID to delete: ");
    String id = sc.nextLine();

    int index = findStudentIndex(id);

    if (index != -1) {
        Student s = students.get(index);
        System.out.println("Student found:");
        s.displayStudent(); // show info before deletion

        System.out.print("Confirm deletion? (Y/N): ");
        String confirm = sc.nextLine();

        if (confirm.equalsIgnoreCase("Y")) {
            students.remove(index); // delete student
            System.out.println("Student deleted successfully!");
            viewAllStudents(); // Show updated list
        } else {
            System.out.println("Deletion cancelled.");
        }
    } else {
        System.out.println("Student not found.");
    }
}
```

Explanation:

- User enters student ID.
- If ID not found → "Student not found."
- If found → system displays student info and asks for confirmation (Y/N).
- User confirms deletion → student removed from list.

Outputs:

- **Student Found and Deleted:**

```
--- SLMS SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Add Student
7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 9
Enter Student ID to delete: BSW01085973
Student found:
Student ID   : BSW01085973
Name        : Nur Dayah Sarwani
Email       : dayah@gmail.com
Phone      : 0167542987
-----
Confirm deletion? (Y/N): Y
Student deleted successfully!

--- All Students ---
Student ID   : BSW01085928
Name        : Wan Nur Sabrina
Email       : sabrina@gmail.com
Phone      : 0123456678
-----
```

- **Student ID Not Found:**

```
Enter Student ID to delete: bsw01098786
Student not found.
```

Good Practices Observed:

- Confirms deletion to prevent accidental removal.
- Case-insensitive confirmation input (y/Y) works.
- Clear feedback for both success and failure.

9.6 VIEW ALL STUDENTS FUNCTION

The **View All Students** function displays all student records stored in the system.

Implementation:

- The system loops through the **ArrayList** of students.
- The **displayStudent()** method is called for each student.

Before displaying, the system checks whether the list is empty to prevent errors. This ensures student information is shown in a **clear and organized format**.

```
public void displayStudent() {  
    System.out.println("Student ID   : " + studentID);  
    System.out.println("Name       : " + firstName + " " + lastName);  
    System.out.println("Email      : " + email);  
    System.out.println("Phone     : " + phone);  
    System.out.println("-----");  
}
```



```
--- All Students ---  
Student ID   : BSW01085928  
Name        : Wan Nur Sabrina  
Email       : sabrina@gmail.com  
Phone       : 0123456678  
-----  
Student ID   : BSW01085973  
Name        : Hidayah Sarwani  
Email       : dayah@gmail.com  
Phone       : 0167542987  
-----
```

Good Practices Observed:

- **Reusability:** Uses displayStudent() instead of repeating print statements.
- **Validation:** Handles empty list gracefully.
- Checks if the list is empty.

9.7 EXTRA INPUT HANDLING

- Lowercase IDs still work due to equalsIgnoreCase().
- Example: Searching for bsw01085976 still finds the student.

LOWERCASE ID:

```
Enter Student ID to search: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Ahmad
Email       : ilyass@gmail.com
Phone       : 0198765543
-----
```

LOWERCASE CONFIRMATION:

```
Choose an option: 9
Enter Student ID to delete: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Irfan
Email       : ilyass@gmail.com
Phone       : 0198765563
-----
```

```
Confirm deletion? (Y/N): y
Student deleted successfully!
```

```
--- All Students ---
Student ID   : BSW01085928
Name        : Wan Nur Sabrina
Email       : sabrina@gmail.com
Phone       : 0123456678
-----
```

Good Practices:

- All fields validated after input.
- User-friendly: all fields input first without interruption.
- Clear feedback: success/failure at the end.
- Encapsulation: Student attributes remain private, added via constructor.

10.0 COURSEMANAGER CLASS

10.1 COURSEMANAGER CLASS DESIGN

- **Class declaration:**



```
class CourseManager {
```

Explanation:

- This is the controller for Course class.
- It manages:
 - Course data
 - Student data
 - Relationship between students and courses
- It acts as the core logic handler for the SLMS (Student Learning Management System).

- **Attributes:**

```
35 // Stores last searched student ID (for history feature)
36 private String lastSearchedStudent = "";
37
38 // Stores last searched course code (for history feature)
39 private String lastSearchedCourse = "";
40
41 // ArrayList to store all courses. ArrayList allows dynamic resizing.
42 private ArrayList<Course> courses = new ArrayList<>();
43
44 // ArrayList to store all students. Dynamic insertion and deletion possible.
45 private ArrayList<Student> students = new ArrayList<>();
46
47 // Stores search history for API caching feature
48 private ArrayList<String> searchCache = new ArrayList<>();
49
50 /// Relationship matrix:
51 // enrollment[studentIndex][courseIndex] = true if student is enrolled
52 private boolean[][] enrollment = new boolean[100][100];
```

Explanation:

1. Courses

- courses courses is an ArrayList used to store all Course objects. Each course contains Course name, Course code, Credit hour, Summary, Link and why use ArrayList because of, dynamic size, Easy to add or remove data, Easy searching and editing, More flexible than normal arrays, This helps manage course records efficiently.

```
// Add when all fields are valid
courses.add(new Course(name, code, credit, summary, link));
System.out.println(x: "Course added successfully!");

// Remove course from list
courses.remove(index);
```

2. Students

- The students attribute is an ArrayList used to store all Student objects in the system. Each student object contains information such as student ID, first name, last name, email, and phone number. The use of ArrayList allows the system to add, update, or remove student records dynamically. This makes the system scalable and efficient as the number of students increases.

```
// Only add when all fields are valid
students.add(new Student(id, fname, lname, email, phone));
System.out.println(x: "Student added successfully!");

// Clear relationships first
students.remove(index);
System.out.println(x: "Student deleted successfully!");
```

3. SearchCache

- searchCache is used to store previous search history as a simple caching mechanism. Instead of performing the same search repeatedly from scratch, the system can reuse stored search data to improve speed and efficiency. This reduces redundant processing and helps users quickly access previously searched students or courses. Search caching improves performance because frequently accessed search results can be recalled faster, and it also enhances user convenience through search history functionality. This feature contributes to a smoother and more responsive system.

4. Enrollment

- Represents:

```
enrollment[sIndex][cIndex]
```

- enrollment is a two-dimensional boolean array (relationship matrix) used to manage the relationship between students and courses. Each row usually represents a student, while each column represents a course. The value inside the matrix determines enrollment status:
 - true → The student is enrolled in that course
 - false → The student is not enrolled in that course

10.2 RELATIONSHIP MANAGEMENT FUNCTION DEVELOPMENT

10.2.1 ASSIGN COURSE TO STUDENT FUNCTION

```
613     private void assignCourseToStudent() {
614         System.out.print("Enter Student ID: ");
615         String studentId = sc.nextLine();
616         int sIndex = findStudentIndex(studentId);
617
618         if (sIndex == -1) {
619             System.out.println("Error: Student not found.");
620             return;
621         }
622
623         System.out.print("Enter Course Code: ");
624         String courseCode = sc.nextLine();
625         int cIndex = findCourseIndex(courseCode);
626
627         if (cIndex == -1) {
628             System.out.println("Error: Course not found.");
629             return;
630         }
631
632         if (enrollment[sIndex][cIndex]) {
633             System.out.println("Error: Student already enrolled in this course.");
634             return;
635         }
636
637         // if NOT enrolled yet → assign
638         enrollment[sIndex][cIndex] = true;
639         System.out.println("Success: Course assigned to student.");
640     }
```

Explanation:

- assignCourseToStudent() is a function used to connect a student with a course by updating the enrollment matrix. The user selects a student and a course, and the system checks whether both records exist before assignment.
- Once validated, the corresponding position in the enrollment matrix is changed from false to true, indicating that the student is now enrolled in that course. Validation helps prevent invalid assignments, such as assigning non-existent students or courses, making the system more reliable and accurate.
- This method assigns a course to a student by creating a relationship between them.

- The user enters a student ID and a course code as input.

```
Choose an option: 11
Enter Student ID: sw01083600
Enter Course Code: CISB3323
Success: Course assigned to student.
```

- The program first checks whether the student exists using the `findStudentIndex()` method.

```
589     private int findStudentIndex(String id) {
590         for (int i = 0; i < students.size(); i++) {
591             // equalsIgnoreCase: allows case-insensitive comparison
592             if (students.get(i).getStudentID().equalsIgnoreCase(id)) {
593                 return i; // student found, display student's information
594             }
595         }
596     }
```

- If the student is not found, an error message is displayed and the process stops.

```
596         // -1 indicates student not found
597         return -1;
598     }
```

- Next, the program checks whether the course exists using the `findCourseIndex()` method.

```
373     private int findCourseIndex(String code) {
374         for (int i = 0; i < courses.size(); i++) {
375             // equalsIgnoreCase: ignores uppercase/lowercase differences
376             if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {
377                 return i; // Course found
378             }
379         }
380     }
```

- If the course is not found, an error message is displayed.

```
380         // -1 indicates course not found
381         return -1;
382     }
```

- The system then verifies whether the student is already enrolled in the selected course by checking the enrollment matrix.

```
51     // enrollment[studentIndex][courseIndex] = true if student is enrolled
52     private boolean[][] enrollment = new boolean[100][100];
```

- If the student is already enrolled, the system prevents duplication and shows an error message.

```
Choose an option: 11
Enter Student ID: SW01083600
Enter Course Code: CISB3323
Error: Student already enrolled in this course.
```

- If all validations pass, the system updates the enrollment matrix by setting the value to true. This indicates that the student is successfully assigned to the course.

10.2.2 LIST COURSES BY STUDENT FUNCTION

Explanation:

- listCourseByStudent() displays all courses assigned to a specific student. After the user enters a valid student ID, the system checks the enrollment matrix row for that student and identifies all columns marked true. Each true value means the student is enrolled in that particular course,
- This method displays all courses assigned to a specific student.

```
649     private void listCoursesByStudent() {
650         System.out.print("Enter Student ID: ");
651         String id = sc.nextLine();
652         int sIndex = findStudentIndex(id);
653
654         if (sIndex == -1) {
655             System.out.println("Error: Student not found.");
656             return;
657         }
658
659         boolean found = false;
660
661         for (int j = 0; j < courses.size(); j++) {
662             if (enrollment[sIndex][j]) {
663                 courses.get(j).displayCourse();
664                 found = true;
665             }
666         }
667
668         if (!found) {
669             System.out.println("This student has no courses.");
670         }
671     }
```

- The user enters a student ID as input.

```
Choose an option: 12
Enter Student ID: sw01083600
```

- The program uses the findStudentIndex() method to locate the student.

```

589     private int findStudentIndex(String id) {
590         for (int i = 0; i < students.size(); i++) {
591             // equalsIgnoreCase: allows case-insensitive comparison
592             if (students.get(i).getStudentID().equalsIgnoreCase(id)) {
593                 return i; // student found, display student's information
594             }
595         }
596         // -1 indicates student not found
597         return -1;
598     }

```

- If the student does not exist, an error message is displayed.

```

Choose an option: 12
Enter Student ID: sw01083400
Error: Student not found.

```

- If the student exists, the program loops through all courses using a for loop. For each course, the system checks the enrollment matrix to determine whether the student is enrolled. If the value is true, the course details are displayed using the displayCourse() method.

```

Choose an option: 12
Enter Student ID: sw01083600
Course Name   : HCI
Course Code   : CISB3323
Credit Hour   : 3
Summary       : Human Computer Interaction
MS Teams Link: teams.hci
-----
Course Name   : AI
Course Code   : CISB3433
Credit Hour   : 3
Summary       : Artificial Intelligence
MS Teams Link: teams.ai
-----
Course Name   : SCM
Course Code   : CISB3533
Credit Hour   : 3
Summary       : Software Construction Management
MS Teams Link: teams.scm
-----

```

10.2.3 LIST STUDENTS BY COURSE FUNCTION

```
681 private void listStudentsByCourse() {
682     System.out.print("Enter Course Code: ");
683     String code = sc.nextLine();
684     int cIndex = findCourseIndex(code);
685
686     if (cIndex == -1) {
687         System.out.println("Error: Course not found.");
688         return;
689     }
690
691     boolean found = false;
692
693     for (int i = 0; i < students.size(); i++) {
694         if (enrollment[i][cIndex]) {
695             students.get(i).displayStudent();
696             found = true;
697         }
698     }
699
700     if (!found) {
701         System.out.println("No students enrolled in this course.");
702     }
703 }
```

Explanation:

- listStudentByCourse() shows all students enrolled in a selected course. The user enters a course code, and the system checks the corresponding course column in the enrollment matrix. Every row marked true indicates a student enrolled in that course.
- The system then displays the student details for each enrolled student. This feature is useful for lecturers or administrators who need to monitor course participation, attendance, or class lists. It provides an efficient way to manage course membership and ensures quick access to student enrollment information.

- The program calls the findCourseIndex() method to locate the course.

```

373     private int findCourseIndex(String code) {
374         for (int i = 0; i < courses.size(); i++) {
375             // equalsIgnoreCase: ignores uppercase/lowercase differences
376             if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {
377                 return i; // Course found
378             }
379         }

```

- If the course is not found, an error message is displayed.

```

Choose an option: 13
Enter Course Code: csnb5233
Error: Course not found.

```

- If the course exists, the program loops through all students using a for loop. For each student, the system checks the enrollment matrix to verify if the student is enrolled in the course.
- If the value is true, the student details are displayed using the displayStudent() method.

```

Choose an option: 13
Enter Course Code: cisb3433
Student ID   : SW01083600
Name        : Abu Ali
Email       : abu@gmail.com
Phone      : 019-9099876
-----
Student ID   : sw01083500
Name        : aminah ali
Email       : aminah@gmail.com
Phone      : 01988776655
-----
Student ID   : sw01083700
Name        : said ali
Email       : said@gmail.com
Phone      : 01344556677
-----

```

- A boolean variable is used to check whether any students are enrolled. If no students are found, the system displays the message “No students enrolled in this course.”

```
Choose an option: 13
Enter Course Code: csnb3333
No students enrolled in this course.
```

```
Choose an option: 14
No search history yet.
```

11.0 SYSTEM TESTING

11.1 SCENARIO 1

```
Choose an option: 1
Enter Course Name: SCM
Enter Course Code: cisb3333
Enter Credit: 3
Enter Summary: coding and github
Enter Link: scm.teams
Course added successfully!
```

- Added the courses successfully.

```
Choose an option: 3
Enter Course Code to edit: cisb3323
Course not found.
```

- Entered the wrong option on the menu. The system prompts the user to choose between the valid options.

```
Choose an option: cisb3333
Invalid input! Enter number: 3
Enter Course Code to edit: cisb3333
Editing Course: cisb3333
New Course Name: software cm
New Credit Hour: 3
New Summary:
Summary cannot be empty!
New Summary: code in an efficient way
New MS Teams Link: scm.link.teams
Course updated successfully!
```

- When editing the course, the field for “course summary” was left blank. The system prompts the user to re-enter input. Once all fields have been filled, the new and edited course attributes will be saved according to the course’s code.

```
Course updated successfully!
Course Name : software cm
Course Code : cisb3333
Credit Hour : 3
Summary      : code in an efficient way
MS Teams Link: scm.link.teams
-----
```

- The system will display the edited course to user after successfully editing the course.

11.2 SCENARIO 2

```
Choose an option: 1
Enter Course Name: hci
Enter Course Code: cisb3333
Enter Credit: 3
Enter Summary: hci
Enter Link: hci
Course added successfully!
```

- Added a course successfully.

```
Choose an option: 4
Enter Course Code to delete: cisb2333
Course not found.
```

- Entered non-existing course code, the system displays the following error message.

```
Confirm deletion? (Y/N): d
Invalid input! Please enter Y or N only.
Confirm deletion? (Y/N): y
Course deleted successfully!
```

- User is asked for confirmation to delete. When entered inputs other than “y” or “n”, the system prompts the user to re-enter either “y” or “n” only. Once entered the correct input, the system will delete the course and display a successful deletion message.

11.3 SCENARIO 3

```
Choose an option: 1
Enter Course Name: scm
Enter Course Code: cisb3333
Enter Credit: 3
Enter Summary: scm
Enter Link: scm.link
Course added successfully!
```

- Added a course successfully.

```
Choose an option: 6
Enter Student ID: sw01
Enter First Name: abu
Enter Last Name: ali
Enter Email: abu@mail
Enter Phone: 01234
Student added successfully!
```

- Added a student successfully.

```
Choose an option: 11
Student ID: sw02
Course Code: cisb3333
Invalid student/course.
```

- When trying to assign course to student, if the user entered any non-existing course/student, the system would display the following error message.

```
Choose an option: 11
Student ID: sw01
Course Code: cisb3333
Success: Course assigned to student.
```

- When entered the correct and existing student and course, the system will display the following success message.

```
Choose an option: 12
Student ID: sw02
Error : Student not found.
```

- When trying to view student's courses, if user entered the non-existing student, the system would display the following error message.

```
Choose an option: 12
Student ID: sw01
Course Name : scm
Course Code : cisb3333
Credit Hour : 3
Summary : scm
MS Teams Link: scm.link
```

- When entered the correct and existing student, the system will successfully display the all the courses assigned to student.

```
Choose an option: 13
Course Code: cisb3323
Error : Course not found.
```

- When trying to students in a course, the system will prompt the user to enter the course code. If user entered non existing code, the system would display the following error message.

```
Choose an option: 13
Course Code: cisb3333
Student ID : sw01
Name : abu ali
Email : abu@mail
Phone : 01234
```

- When user entered an existing course code, the system will successfully display all students in the chosen course.

12.0 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES AND CHALLENGES

12.1 BEST PRACTICES APPLIED

Several software construction best practices have been effectively applied during the system testing and finalization phase of the Student Learning Management System (SLMS).

Firstly, unit testing is systematically performed to ensure that individual functions operate correctly. Methods such as `addCourse()`, `searchCourse()`, `editCourse()`, `deleteCourse()`, and `addStudent()` are tested independently using different input scenarios. This practice helps in identifying bugs early and ensures that each function behaves according to its intended design.

Secondly, integration and system testing are carried out to verify the interaction between modules. The collaboration between `CourseManager` and `StudentManager`, particularly in functions such as `assignCourseToStudent()`, `listCoursesByStudent()`, and `listStudentsByCourse()`, demonstrates proper integration. This ensures that data flows correctly between components and that the system works as a complete unit.

In addition, defensive programming and error handling are strongly implemented. The system validates user inputs, prevents invalid operations, and handles error-prone scenarios such as empty fields, incorrect data types, duplicate entries, and non-existent records. Clear and user-friendly error messages are provided, which enhances usability and system robustness.

Another important practice is code readability and maintainability. The system follows consistent coding standards, including proper indentation, meaningful variable names, and clear method structure. Code comments and documentation headers are included to explain the functionality of classes and methods, making the code easier to understand and maintain.

Furthermore, documentation and version control are properly managed. The system includes structured documentation that explains design decisions, module descriptions,

and system behavior. The use of a GitHub repository ensures proper version tracking, supports collaboration, and reflects professional software development practices.

12.2 CHALLENGES FACED

Several challenges were encountered during the system testing and finalization phase.

One of the main challenges was designing effective test cases to cover both normal and error-prone scenarios. Ensuring that all possible user inputs, including invalid and edge cases, are properly tested required careful planning and multiple iterations of testing.

Another challenge was debugging errors identified during unit and system testing. Some issues only appeared when multiple modules interacted, making them harder to trace. This required careful analysis of program flow and data interaction between classes.

Ensuring consistent error handling across all modules was also challenging. Different methods required different types of validation, such as checking for empty inputs, numeric validation, and duplicate detection. Maintaining consistency in how errors are handled and messages are displayed required additional effort.

In addition, maintaining code readability while adding testing logic and error handling increased the complexity of the code. Extra care was needed to ensure that the system remains well-organized and easy to understand despite the added functionality.

Finally, preparing proper documentation was also a challenge. Clearly explaining system behavior, testing results, and design decisions in a structured manner required time and attention to detail to ensure completeness and

clarity.

12.3 CONCLUSION

In conclusion, the system testing and finalization phase successfully demonstrates the application of important software construction best practices, including unit testing, integration testing, defensive programming, and proper documentation. Although several challenges were encountered, they were effectively addressed through careful testing, debugging, and code refinement. The result is a well-tested, reliable, and maintainable system that meets the required evaluation criteria.

13.0 EVALUATION CRITERIA

The system testing and finalization phase ensures that the Student Learning Management System (SLMS) operates correctly, efficiently, and meets all required software construction standards. This phase integrates both testing activities from Lab 9 and documentation and code quality improvements from Lab 10 into a unified evaluation.

During the unit testing phase, individual methods from modules such as CourseManager and StudentManager were tested independently to verify their correctness. Core functionalities including addCourse(), searchCourse(), editCourse(), deleteCourse(), and addStudent() were evaluated using various input scenarios. These tests helped identify issues such as duplicate entries, invalid input formats, and incorrect data handling. By detecting and resolving these issues early, the system ensures that each function performs reliably according to its intended purpose.

System testing was conducted to validate the integration between modules. The interaction between CourseManager and StudentManager, particularly through functions such as assignCourseToStudent(), listCoursesByStudent(), and listStudentsByCourse(), was thoroughly tested. The results confirm that the system successfully manages relationships between students and courses and maintains consistent data flow across components. This demonstrates that the system is fully integrated and functions as a cohesive unit.

The system also demonstrates strong defensive programming and error handling capabilities. Various error-prone scenarios were tested, including empty input fields, invalid numeric values, duplicate identifiers, and non-existent student or course

records. The system effectively prevents invalid operations and provides clear, user-friendly error messages. Input validation techniques such as checking for empty values and validating numeric inputs ensure data integrity and reduce the risk of runtime errors.

From a code quality perspective, the system follows good software construction practices. The program is modular, with separate classes such as `Course`, `Student`, `CourseManager`, and `StudentManager` handling specific responsibilities. Encapsulation is properly implemented by restricting direct access to attributes and using getter and setter methods. This improves maintainability and ensures better control over data manipulation.

Readability is enhanced through consistent coding standards, meaningful naming conventions, and proper indentation. Additionally, code comments and documentation headers are included to clearly describe the purpose of each class and method. This makes the system easier to understand and maintain for future development.

The finalization phase also includes proper documentation of system design and implementation. Key design decisions, data structures (such as `ArrayList` and 2D arrays), and module functionalities are clearly described. The integration of all components into a single menu-driven interface demonstrates a complete and well-structured system architecture.

Furthermore, the project is maintained in a GitHub repository, ensuring proper version control and tracking of updates. This reflects good software engineering practices and supports collaboration and project organization.

Overall, the SLMS system successfully meets the evaluation criteria for both Lab 9 and Lab 10. The system is functionally correct, well-tested, robust against invalid inputs, and adheres to high standards of code quality and documentation. The final product is reliable, maintainable, and demonstrates a strong understanding of software construction principles.